Smart Retail Customer Analysis & Prediction

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

S. Niteesh	**:2420030178**
Gagan	**:2420030352**
P. Nitish	**:2420030642**

Under the Esteemed Guidance of

Ms. Prathusha

Assistant Professor

Department of Computer Science and Engineering

AY- 2026-2027

Semester: III Year - I Semester



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**Smart Retail Customer Analysis & Prediction**" is a record of Bonafide work of **S. Niteesh – 2420030178, Gagan – 2420030352, P. Nitish – 2420030642** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

S. Niteesh – 2420030178

Gagan – 2420030352

P. Nitish – 2420030642

K L Deemed to be University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled "**Smart Retail Customer Analysis & Prediction**" is a bonafide work done and submitted by **S. Niteesh – 2420030178, Gagan – 2420030352, P. Nitish – 2420030642** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L Deemed to be University, during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Nalajala Sunanda**, and my guide **Ms. Prathusha,** Department of CSE which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **P. Venkateswara Rao**, Head of the ***Department for Computer Science and Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

S. Niteesh – 2420030178

Gagan – 2420030352

P. Nitish – 2420030642

Abstract

Retail businesses generate large volumes of raw, unorganised transaction and customer data that is rarely usable in its original form. In this project, raw data collected from retail transactions (customer ID, purchase date, product category, quantity, price, etc.) is first cleaned and converted into structured data through pre-processing steps such as handling missing values, removing duplicates and outliers, correcting data types, and normalising fields. This cleaned data is then organised into structured tables/features (e.g., Recency, Frequency, and Monetary value per customer) using Pandas and NumPy, transforming scattered raw records into an analysis-ready dataset. This structured data is analysed to identify purchasing patterns, product preferences, and spending behaviour, forming the foundation for customer segmentation and prediction.

Using this structured data, K-Means clustering (Scikit-learn) is applied to segment customers into groups such as high-value, frequent, and at-risk customers, while classification/regression models (Logistic Regression, Random Forest, XGBoost) predict future purchase likelihood and customer value. The complete solution is built in Python, using Pandas/NumPy for data structuring, Matplotlib/Seaborn for visualization, Scikit-learn for machine learning, MySQL/SQLite for data storage, and Streamlit/Power BI for an interactive dashboard where retailers can view customer insights, trends, and predictions. By converting raw retail data into structured, analysis-ready information and applying predictive analytics on it, the system aims to improve customer engagement, personalised marketing, and sales planning, resulting in a smarter, more efficient retail management system.

Keywords: Customer segmentation, RFM analysis, K-Means clustering, purchase prediction, retail analytics, Streamlit.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience 1.7 Literature survey 1.8 Problem gap identification 1.9 Innovation, creativity and novelty 1.10 Feasibility and project plan	1-6
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	7-8
3	Technology Stack	3.1 Front-end (Streamlit / Power BI) 3.2 Back-end (Python data and ML layer) 3.3 Database (MySQL / SQLite) 3.4 Version control (Git, GitHub) 3.5 APIs or external services	9-10
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	11-13
5	Design	5.1 Wireframes or mockups 5.2 ER diagram / database schema 5.3 Data-flow diagram	14-16
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 Backend logic (modules and business logic) 6.4 Database connectivity 6.5 API integration 6.6 Authentication & authorization	17-20
7	Features	7.1 List of main features 7.2 How each feature works	21-22
8	Testing	8.1 Functional testing of front-end and back-end (Postman not required) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	23-24
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of front-end and back-end	25-26
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	27-28
11	Future Enhancements	11.1 Planned features	29

		11.2 Possible integrations or optimizations	
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	30-31
13	References	- Books, tutorials, APIs, documentation sites used	32
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Certifications	33-34

CHAPTER 1
Introduction

1.1 Background of the project

Retail businesses generate large volumes of transaction and customer data every day: invoices, purchase dates, product categories, quantities and prices. In its original form this data is raw and unorganised, and it is rarely usable for decision making. Records contain missing values, duplicates, wrongly typed fields and outliers, and the information about any single customer is scattered across thousands of rows.

Customer analytics addresses this problem by converting raw records into structured, customer-level features and then applying data-mining and machine-learning techniques on them. A widely used representation is **RFM analysis**, in which every customer is described by **Recency** (how recently the customer purchased), **Frequency** (how often the customer purchases) and **Monetary value** (how much the customer spends). Once customers are described by such features, **K-Means clustering** can group them into meaningful segments, and supervised models can estimate how likely each customer is to buy again.

This project, **Smart Retail Customer Analysis & Prediction**, builds such a pipeline end to end in Python. Raw retail transaction data is cleaned and pre-processed, organised into analysis-ready tables with Pandas and NumPy, segmented with K-Means (Scikit-learn), and used to train Logistic Regression, Random Forest and XGBoost models that predict future purchase likelihood and customer value. The results are stored in MySQL/SQLite and presented through an interactive Streamlit / Power BI dashboard so that retailers can view customer insights, trends and predictions without any technical background.

1.2 Problem statement

Retailers collect large amounts of customer and transaction data, but this data is often not converted into useful insights. Manual analysis becomes difficult

when thousands of customers are involved, because purchase frequency, spending and product preferences are complex to understand by inspection. Without proper customer segmentation, retailers tend to send the same promotion to everyone, which makes marketing less targeted and less effective.

A second major challenge is the lack of visibility into future customer purchases: it is hard for a retailer to know who is likely to buy again and who is about to be lost. Therefore, this project uses data analysis and machine learning to understand customer behaviour, segment customers into meaningful groups, and predict future purchasing patterns, and it delivers the outcome through a simple dashboard.

1.3 Scope of the project

The scope of the project covers the complete analytics workflow for a retail transaction dataset:

- Uploading a retail transaction dataset in CSV or Excel format and profiling its structure, data types, missing and unique values.
- Pre-processing: handling missing values, removing duplicates and outliers, correcting data types and normalising fields.
- Feature engineering: Recency, Frequency and Monetary value per customer, extended with product-category diversity and purchase-frequency trend.
- Customer segmentation using K-Means clustering, with the optimal number of clusters selected by the Elbow method.
- Prediction of future purchase likelihood and customer value using Logistic Regression, Random Forest and XGBoost, and comparison of the models using Accuracy, Precision, Recall and F1-score.
- Storage of structured data and results in MySQL/SQLite and an interactive Streamlit / Power BI dashboard for retailers.

The following items are **outside the scope** of the present work: real-time integration with point-of-sale (POS) systems, automatic sending of campaigns (e-mail/SMS), product recommendation engines and payment processing. Some of these are listed as future enhancements in Chapter 11.

1.4 Objective(s)

1. **Analyse customer behaviour:** study transactional data (purchase history, frequency, spending and product preferences) to uncover behavioural patterns.
2. **Segment customers meaningfully:** group customers using clustering techniques into segments such as high-value, frequent and price-sensitive buyers.
3. **Predict future purchases:** build a machine-learning model that estimates the likelihood and timing of a customer's next purchase.
4. **Deliver actionable insights:** provide a simple dashboard so that retailers can view segments, trends and predictions without technical expertise.
5. **Enable data-driven decisions:** support personalised marketing, better sales planning and improved customer-engagement strategies.

1.5 Importance of the application

- **Targeted marketing:** different segments receive different offers instead of one common promotion, which improves the relevance of campaigns and avoids wasted marketing effort.
- **Customer retention:** at-risk customers are identified early, so that retention actions can be taken before they are lost.
- **Revenue focus:** high-value customers are flagged so that the retailer can protect and grow the most profitable part of the customer base.
- **Sales planning:** purchase predictions and trends support inventory, staffing and promotion planning.
- **Accessibility:** most advanced analytics tools are costly and complex; this solution is built entirely from open-source tools and is designed for small and medium retail businesses.

1.6 Target users/audience

- Store managers and retail owners who need a quick view of customer groups and trends.

- Marketing teams who need customer lists for targeted promotions and retention campaigns.
- Business and data analysts who want to explore the data, features and model results.
- Small and medium retail businesses that cannot afford enterprise analytics suites.
- Students and faculty evaluating the project as part of the Data Science course (24AD3107).

1.7 Literature survey

Three recent studies on RFM-based customer segmentation were reviewed. Their methodology, inferences and limitations are summarised in Table 1.1.

Authors / Title / Year	Methodology	Inferences	Limitation / Future work
Mahfuza et al. [1] LRFMV: An Efficient Customer Segmentation Model for Superstores, PLOS ONE, 2024	Extends RFM with Length and Volume (LRFMV); K-Means, K-Medoids and Mini-Batch K-Means on superstore transaction data.	LRFMV segments show a clearer profit-to-quantity relationship than RFM/LRFM; K-Means gave the most precise clusters.	Volume feature is specific to superstore data; does not address predicting future purchases.
Ullah et al. [2] Customer Analysis Using ML-Based Classification for Segmentation Using RFMT, Sensors, 2025	RFMT (Recency, Frequency, Monetary, Time) model with cluster validation and majority voting across several algorithms on e-commerce data.	Adding the Time dimension improved cluster validity and segmentation accuracy over standard RFM.	Focused on segmentation only; no forward-looking prediction of purchase likelihood.
Chavhan et al. [3] RFM Analysis for Customer Segmentation Using Machine Learning: A Survey of a Decade of Research, 2026	Comparative survey of RFM-based clustering (K-Means, hierarchical, fuzzy C-means) on online retail datasets.	Fuzzy C-Means gave the best silhouette scores but at higher computational cost than K-Means.	Survey-level work; no end-to-end segmentation-plus-prediction system for retailers.

Table 1.1 Literature survey summary

1.8 Problem gap identification

1. **Narrow feature set:** most RFM-based methods use only Recency, Frequency and Monetary value and ignore product-category diversity and preference trends.
2. **Segmentation without prediction:** many studies stop at descriptive clustering and do not connect segments to a forward-looking model of future purchases.
3. **Not built for everyday retail use:** advanced analytics tools are often costly, complex and not tailored to small and medium retail businesses.
4. **No unified, interpretable output:** few solutions combine segmentation and prediction in one simple dashboard that non-technical retail staff can act on.

1.9 Innovation, creativity and novelty

- **Unified segmentation + prediction pipeline:** customer segmentation and purchase prediction are combined in one workflow instead of being separate, disconnected tasks.
- **Extended behavioural features:** goes beyond standard RFM by adding product-category diversity and purchase-frequency trend.
- **Retailer-friendly dashboard:** insights and predictions are presented in a simple visual interface designed for users without a data-science background.
- **Action-oriented outputs:** directly usable outputs such as targeted promotion lists, at-risk customer alerts and high-value customer flags rather than raw statistics.

1.10 Feasibility and project plan

The project is technically feasible because every component (Python, Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib/Seaborn, SQLite/MySQL and Streamlit) is open source and well documented. It is economically feasible because no paid software or licence is required (Power BI Desktop is optional

and free), and operationally feasible because the dashboard needs no programming knowledge from the end user. The work is organised in six phases over 13 weeks (Table 1.2).

Phase	Activity	Duration
Phase 1	Literature survey & problem identification	Weeks 1-2
Phase 2	Data collection, cleaning & exploratory analysis	Weeks 3-4
Phase 3	Customer segmentation (clustering)	Weeks 5-6
Phase 4	Predictive model development & tuning	Weeks 7-9
Phase 5	Dashboard / interface development	Weeks 10-11
Phase 6	Testing, evaluation & final report	Weeks 12-13

Table 1.2 Project timeline

CHAPTER 2
System Requirements

2.1 Hardware requirements

The system runs on an ordinary student or small-business computer; no GPU is required because the datasets involved are tabular and the models (K-Means, Logistic Regression, Random Forest, XGBoost) train comfortably on a CPU.

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (dual core)	Intel Core i5 / AMD Ryzen 5 or higher (quad core)
RAM	4 GB	8 GB or more
Storage	5 GB free disk space	10 GB free space on SSD
Display	1366 x 768	1920 x 1080 (for dashboard viewing)
Network	Required for installing packages	Broadband connection (for cloud deployment)

Table 2.1 Hardware requirements

2.2 Software requirements

Category	Requirement
Operating system	Windows 10/11, Ubuntu 22.04 or later, or macOS
Programming language	Python 3.10 or later
Data & ML libraries	Pandas, NumPy, Scikit-learn, XGBoost, Joblib
Visualisation libraries	Matplotlib, Seaborn (plus built-in Streamlit charts)
Dashboard	Streamlit (primary); Power BI Desktop (optional BI report)
Database	SQLite 3 (default, file based) or MySQL 8.x; SQLAlchemy connector
Web browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest)
Version control	Git and a GitHub account

Table 2.2 Software requirements

2.3 Development tools and frameworks

- **VS Code / Jupyter Notebook / Google Colab:** code development, exploratory analysis and experimentation.
- **Streamlit:** Python framework used to build the interactive dashboard.
- **Scikit-learn and XGBoost:** machine-learning frameworks for clustering, classification/regression and evaluation.

- **MySQL Workbench / DB Browser for SQLite:** inspecting and managing the database.
- **Git and GitHub:** version control and collaboration among the three team members.
- **pytest:** automated testing of data-processing and model functions.

CHAPTER 3

Technology Stack

Technology	Role in the project
Python	Core programming language for the complete pipeline
Pandas	Data manipulation: cleaning, grouping and structuring records into tables
NumPy	Numerical computing and array operations
Scikit-learn	K-Means clustering, Logistic Regression, Random Forest, scaling, metrics
XGBoost	Gradient-boosted trees for purchase prediction
Matplotlib / Seaborn	Static visualisation: Elbow curve, distributions, heat maps
Streamlit / Power BI	Interactive retailer dashboard
MySQL / SQLite	Persistent storage of data, features, segments and predictions

Table 3.1 Tools and technologies used

3.1 Front-end (Streamlit / Power BI)

The user interface is built with **Streamlit**, a Python framework that turns scripts into interactive web applications. It provides a file uploader, sidebar navigation, selection widgets, data tables, chart containers and download buttons, so the whole front-end can be written in Python without HTML, CSS or JavaScript. Charts are drawn with Matplotlib and Seaborn or Streamlit's native chart components. As an alternative presentation layer, the same stored results can be loaded into **Power BI**, which allows retailers who already use Microsoft tools to build their own reports on the segments and predictions.

3.2 Back-end (Python data and ML layer)

The back-end is a set of Python modules rather than a separate web server. **Pandas** and **NumPy** structure the raw data and compute the customer features; **Scikit-learn** provides `StandardScaler`, `KMeans`, `LogisticRegression`, `RandomForestClassifier`, `train_test_split` and the evaluation metrics; **XGBoost** supplies the gradient-boosting model. Trained models are persisted with **Joblib** so that the dashboard can load them without retraining.

3.3 Database (MySQL / SQLite)

Structured data is kept in a relational database. **SQLite** is the default because it is file based and needs no server, which suits a single-user prototype. **MySQL** can be used instead when several users must share the same data. Both are accessed through SQLAlchemy and Pandas (`to_sql` and `read_sql`), so switching between them requires only a change of connection string.

3.4 Version control (Git, GitHub)

The source code, notebooks and documentation are maintained in a Git repository hosted on GitHub. Each team member works on a branch and merges through pull requests, which keeps a history of changes and allows the application to be deployed directly from the repository.

3.5 APIs or external services

The system does not depend on any paid or third-party data API; all analysis is done on the dataset supplied by the user. The only external services used are optional: **Streamlit Community Cloud** for hosting the dashboard, **Power BI Service** for sharing a BI report, and **GitHub** for source hosting.

CHAPTER 4

System Architecture

4.1 High-level architecture diagram

The design of the system follows an eight-step analytics pipeline, shown in Figure 4.1. A user uploads the retail dataset, which is profiled, pre-processed and converted into RFM features; the features feed customer segmentation and prediction modelling; the models are validated and compared; and the outputs (segments, trends, predictions and actionable insights) are shown in an interactive dashboard.

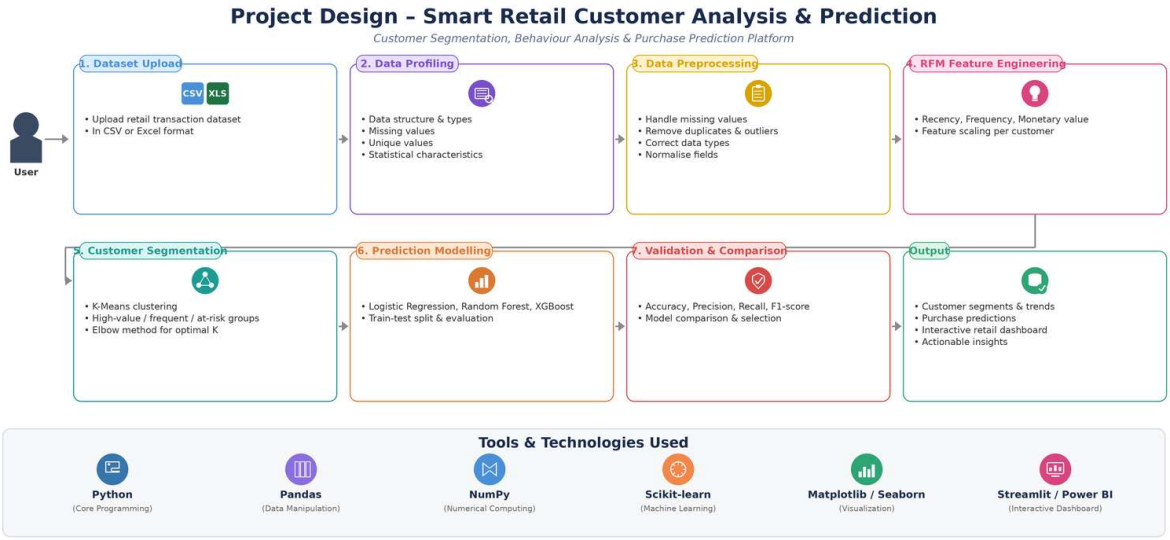


Fig. 4.1 Project design flow (Review-1)

Figure 4.2 shows the same system as a layered architecture, in which each layer communicates only with the layer next to it.

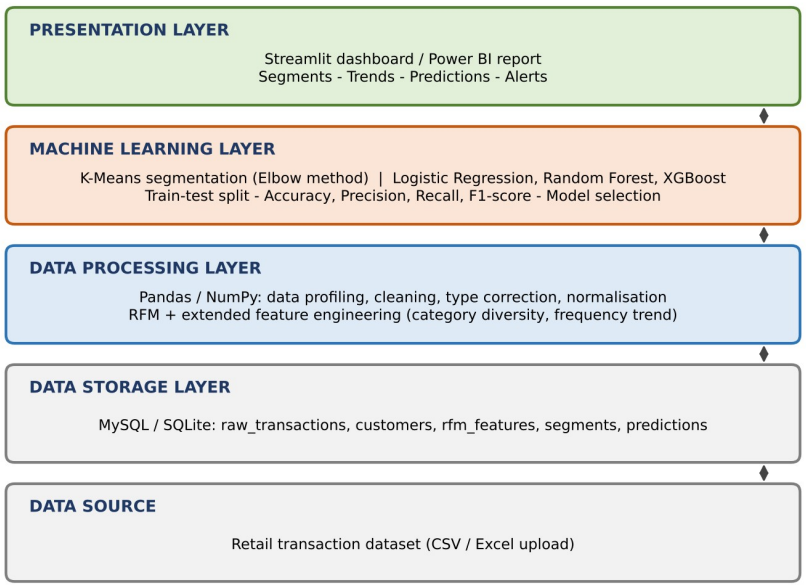


Fig. 4.2 Layered system architecture

4.2 Description of each layer

Layer	Description
Data source	Retail transaction dataset (customer ID, purchase date, product category, quantity, price, etc.) uploaded as CSV or Excel.
Data storage	MySQL/SQLite tables hold the raw transactions, cleaned customers, RFM features, segment labels and predictions.
Data processing	Pandas/NumPy modules perform profiling, cleaning, type correction, normalisation and feature engineering.
Machine learning	K-Means (with Elbow method) for segmentation; Logistic Regression, Random Forest and XGBoost for prediction; metrics-based model comparison.
Presentation	Streamlit dashboard (or Power BI report) showing segments, trends, predictions and action lists for non-technical users.

Table 4.1 Layers of the architecture

4.3 Deployment architecture

The application is developed locally and pushed to GitHub. For use by retailers it can be run on a local server inside the shop or office, or deployed on Streamlit Community Cloud directly from the GitHub repository; the optional Power BI report is published through Power BI Service. Users only need a web browser. An optional CI pipeline (GitHub Actions) can run the automated tests on every push before the application is redeployed.

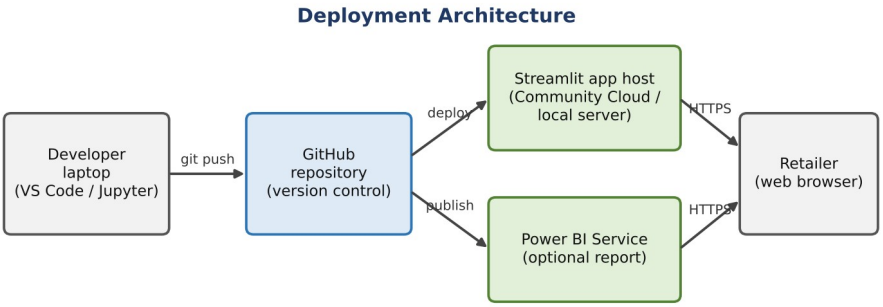


Fig. 4.3 Deployment architecture

CHAPTER 5
Design

5.1 Wireframes or mockups

The dashboard has a sidebar for navigation (Upload Data, Data Profile, RFM Features, Segments, Predictions, Insights) and a main area for results. Wireframe 1 gives the overview with key indicators, a cluster scatter plot, a segment-size chart and a sales-trend chart. Wireframe 2 lets the user choose a model and a segment, shows model metrics and the purchase-probability distribution, and offers the action lists (targeted promotions, at-risk alerts, high-value flags) with a CSV download.

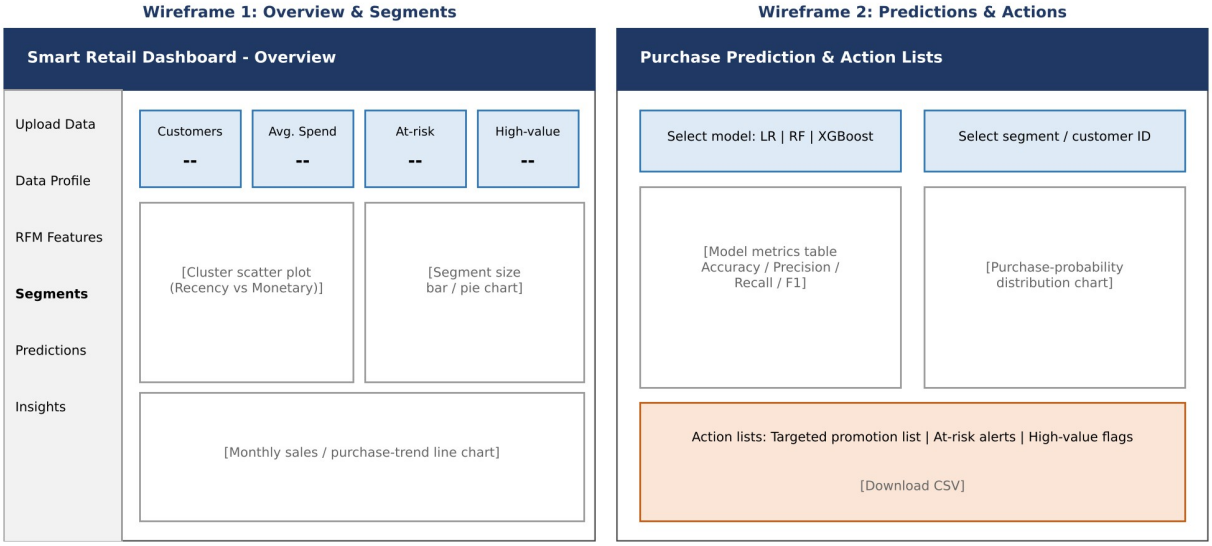


Fig. 5.1 Dashboard wireframes

5.2 ER diagram / database schema

The database stores transactional data and the analytical outputs in six related tables (Figure 5.2). A customer has many transactions, each transaction refers to one product, and every customer has one RFM feature row, one segment assignment and one or more predictions (one per model run).

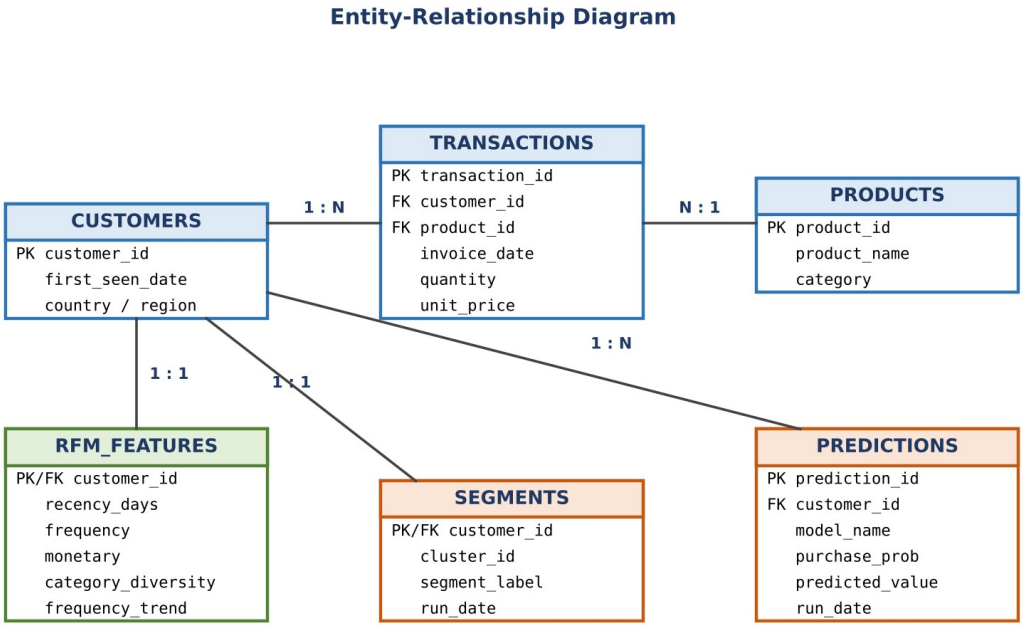


Fig. 5.2 Entity-relationship diagram

Table	Main columns	Purpose
customers	customer_id (PK), first_seen_date, country	One row per customer
products	product_id (PK), product_name, category	Product master with category
transactions	transaction_id (PK), customer_id (FK), product_id (FK), invoice_date, quantity, unit_price	Cleaned purchase records
rfm_features	customer_id (PK/FK), recency_days, frequency, monetary, category_diversity, frequency_trend	Customer-level analysis-ready features
segments	customer_id (PK/FK), cluster_id, segment_label, run_date	K-Means output
predictions	prediction_id (PK), customer_id (FK), model_name, purchase_prob, predicted_value, run_date	Model output per customer

Table 5.1 Database schema

5.3 Data-flow diagram

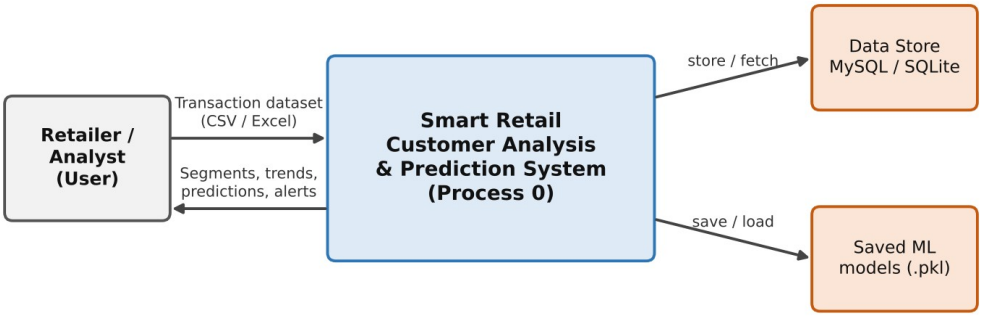


Fig. 5.3 DFD Level-0 (context diagram)

At Level-0 the retailer supplies a transaction dataset to the system and receives segments, trends, predictions and alerts. Internally the system reads and writes the database and saves/loads the trained models.

CHAPTER 6
Implementation

6.1 Module-wise implementation

The implementation follows the modules of the design flow in Figure 4.1.

Module 1: Dataset upload

The user uploads a CSV or Excel file through the Streamlit file uploader. The file is read with `pd.read_csv` or `pd.read_excel` and checked for the required columns (customer ID, invoice/purchase date, product category, quantity and unit price). A clear error message is shown if a required column is missing.

Module 2: Data profiling

The module reports the number of rows and columns, the data type of each field, the count and percentage of missing values per column, the number of unique values and basic statistical characteristics (`describe`). This helps the user understand the quality of the dataset before any change is made.

Module 3: Data pre-processing

- **Missing values:** records without a customer ID are removed; missing categories are labelled as 'Unknown'; missing numeric values are imputed with the median where appropriate.
- **Duplicates:** exact duplicate rows are dropped.
- **Outliers:** extreme quantities and prices are detected with the inter-quartile-range (IQR) rule; cancelled or negative-quantity records are excluded from purchase counts.
- **Data types:** dates are converted with `pd.to_datetime`, identifiers to strings and amounts to numeric types.
- **Normalisation:** `StandardScaler` (or Min-Max scaling) is applied to numeric features before clustering and distance-based models.
- A derived column `TotalAmount = Quantity x UnitPrice` is created for each line.

Module 4: RFM feature engineering

A snapshot date (one day after the last transaction) is chosen and, for each customer, the following features are computed with a Pandas `groupby`:

Feature	Definition
Recency	Number of days between the snapshot date and the customer's most recent purchase
Frequency	Number of distinct purchases (invoices) made by the customer
Monetary	Total amount spent by the customer (sum of TotalAmount)
Category diversity	Number of distinct product categories purchased (extension)
Frequency trend	Change in purchase rate between the recent and earlier part of the observation period (extension)

Table 6.1 Customer features

Module 5: Customer segmentation

The scaled features are clustered with K-Means. The number of clusters K is chosen with the **Elbow method** (plot of within-cluster sum of squares against K), optionally supported by the silhouette score. Each cluster is then given a business label by inspecting its average Recency, Frequency and Monetary values: for example, low Recency with high Frequency and Monetary indicates **high-value** customers, high Recency with low Frequency indicates **at-risk** customers, and customers who buy often but with small baskets form the **frequent / price-sensitive** group.

Module 6: Prediction modelling

Two prediction targets are supported: (i) **purchase likelihood**, a binary label showing whether a customer buys again within a following period, solved with classification, and (ii) **customer value**, the expected future spending, solved with regression. Features are computed from an earlier observation window and the target from the following period, so that no future information leaks into the features. The data is divided into training and test sets (for example 80:20, stratified for classification) and three models are trained: **Logistic Regression** (interpretable baseline), **Random Forest** (non-linear ensemble) and **XGBoost** (gradient boosting). Class weights are used when the classes are imbalanced.

Module 7: Validation and comparison

Classification models are evaluated on the held-out test set using Accuracy, Precision, Recall and F1-score, supported by the confusion matrix; regression models use MAE, RMSE and R-squared. The model with the best balance of F1-score and stability is selected and saved with Joblib. Segmentation quality is judged with the Elbow curve and silhouette score.

Output module

Customer segments and trends, purchase predictions, the interactive dashboard and actionable insights (promotion lists, at-risk alerts and high-value flags) are produced and stored.

6.2 Front-end logic (UI rendering, state management)

The Streamlit app is organised as pages selected from a sidebar (`st.sidebar.radio`). Uploaded and processed DataFrames are kept in `st.session_state` so that they survive page changes and re-runs. Expensive operations such as data loading and model loading are wrapped in `st.cache_data` / `st.cache_resource` to avoid recomputation. Filters (segment, model, date range) are widgets whose values re-render the charts and tables, and `st.download_button` exports the action lists as CSV.

6.3 Backend logic (modules and business logic)

The back-end logic is kept separate from the user interface so that it can be tested and reused. A suggested project layout is shown below.

```
smart_retail/
|-- app.py                # Streamlit entry point (pages)
|-- pipeline/
|   |-- ingest.py         # read + validate uploaded file
|   |-- profile.py        # data profiling
|   |-- preprocess.py     # cleaning, dtypes, outliers, scaling
|   |-- features.py       # RFM and extended features
|   |-- segment.py        # K-Means, Elbow, labelling
|   |-- predict.py        # LR / RF / XGBoost training and prediction
|   `-- evaluate.py       # metrics and model comparison
|-- db.py                 # SQLAlchemy engine, save/load helpers
|-- models/               # saved models (.pkl)
|-- tests/                # pytest test cases
`-- requirements.txt
```

The business rules map each segment to a recommended action: high-value customers are flagged for loyalty rewards, frequent customers for cross-selling,

price-sensitive customers for discount offers and at-risk customers for win-back messages.

6.4 Database connectivity

A single helper in `db.py` creates a SQLAlchemy engine from a connection string, for example `sqlite:///retail.db` for SQLite or `mysql+pymysql://user:password@host/retail` for MySQL. Cleaned data, features, segments and predictions are written with `DataFrame.to_sql` and read back with `pd.read_sql`. Credentials are read from environment variables or Streamlit secrets and are never hard-coded in the source.

6.5 API integration

All modules communicate through Python function calls, so no REST API is needed in the current design. Trained models are persisted with Joblib and loaded by the dashboard. For Power BI, the stored results are made available either through a database connection (MySQL) or through a CSV export. If the system is later extended into a service, the same functions can be exposed through a REST API (for example with FastAPI).

6.6 Authentication & authorization

As the application handles customer data, access is controlled with a simple login. Two roles are planned: an **Analyst** role that can upload data, re-run the pipeline and see all pages, and a **Viewer** role that can only view the dashboard and download action lists. Passwords or tokens are stored in Streamlit secrets, database credentials in environment variables, and customer identifiers can be pseudonymised before they are displayed.

CHAPTER 7
Features

7.1 List of main features

1. Dataset upload (CSV / Excel) with column validation
2. Automatic data profiling report
3. One-click data cleaning and pre-processing
4. RFM and extended feature generation per customer
5. K-Means customer segmentation with Elbow-method support
6. Purchase-likelihood and customer-value prediction using three models
7. Model comparison using Accuracy, Precision, Recall and F1-score
8. Interactive dashboard with segments, trends and predictions
9. Action lists: targeted promotions, at-risk alerts, high-value flags (CSV download)
10. Persistent storage of data and results in MySQL / SQLite

7.2 How each feature works

Feature	User perspective	Technical description
Dataset upload	The user selects a CSV/Excel file and sees a preview.	`st.file_uploader`; `pd.read_csv/read_excel`; required-column check.
Data profiling	Tables show types, missing and unique values.	`info`, `describe`, `isna().sum()`, `nunique()`.
Pre-processing	The user presses one button; a summary of changes is shown.	Missing-value handling, `drop_duplicates`, IQR outlier filter, `to_datetime`, scaling.
RFM features	A customer table with R, F, M and extra features appears.	`groupby("CustomerID").agg(...)` with snapshot date.
Segmentation	Customers appear as coloured groups with names such as "At-risk".	`StandardScaler` + `KMeans`; Elbow plot; centroid-based labelling.
Prediction	The user chooses a model and sees each customer's purchase probability.	`LogisticRegression`, `RandomForestClassifier`, `XGBClassifier`; `predict_proba`.
Model comparison	A table and chart compare the models.	`accuracy_score`, `precision_score`,

Feature	User perspective	Technical description
		`recall_score`, `f1_score`.
Dashboard & action lists	KPIs, charts and downloadable customer lists.	Streamlit widgets, Matplotlib/Seaborn charts, `st.download_button`.

Table 7.1 Working of the features

CHAPTER 8
Testing

8.1 Functional testing of front-end and back-end (Postman not required)

The system is a Python / Streamlit application whose back-end is a set of functions and not a REST service, so API testing with Postman is not needed at present. Instead, back-end functions are tested individually (unit tests) with small, hand-made DataFrames whose correct output is known, and the front-end is tested by running the app and checking each page, widget and download. If the pipeline is later exposed through an API, Postman will be used to test its endpoints.

8.2 Integration testing

Integration tests run the complete chain, upload, profiling, pre-processing, feature engineering, segmentation, prediction, database write and dashboard display, on a sample dataset and verify that the output of each stage is accepted by the next, that the number of customers is preserved from the feature table to the segment and prediction tables, and that results read back from the database equal the results written.

8.3 Tools used for testing

- **pytest:** unit and integration tests of the pipeline functions.
- **pandas.testing:** comparison of expected and actual DataFrames.
- **Scikit-learn metrics:** Accuracy, Precision, Recall, F1-score, confusion matrix, silhouette score.
- **Streamlit AppTest and manual browser testing:** front-end behaviour.
- **GitHub Actions (optional):** automatic test run on every push.

8.4 Test cases and results

Table 8.1 lists the planned test cases with their expected results. The Status column is updated as each test is executed during Phase 6 (Testing, evaluation and final report).

ID	Test case	Input	Expected result	Status
TC-01	Upload valid file	CSV with all required columns	File accepted; preview shown	Planned
TC-02	Upload invalid file	File without CustomerID column	Clear error message; pipeline not started	Planned
TC-03	Missing values	Rows with null CustomerID	Rows removed; count reported	Planned
TC-04	Duplicates	Repeated identical rows	Duplicates dropped	Planned
TC-05	Outliers	Very large quantity / negative quantity	Outliers and returns excluded	Planned
TC-06	RFM calculation	Small dataset with known answer	R, F, M equal hand-computed values	Planned
TC-07	Elbow method	Scaled RFM features	Elbow curve displayed; K selectable	Planned
TC-08	Segmentation	K = chosen value	Every customer gets exactly one segment label	Planned
TC-09	Model training	Train/test split	All three models train; metrics computed	Planned
TC-10	Model comparison	Metrics of three models	Comparison table; best model highlighted	Planned
TC-11	Database round trip	Write then read features	Data read equals data written	Planned
TC-12	Action list export	Select "At-risk" segment	CSV downloads with only at-risk customers	Planned
TC-13	Role access	Viewer login	Upload and re-run pages not accessible	Planned

Table 8.1 Test cases

Evaluation metrics used for the prediction models are: **Accuracy** = (TP + TN) / (TP + TN + FP + FN); **Precision** = TP / (TP + FP); **Recall** = TP / (TP + FN); **F1-score** = 2 x Precision x Recall / (Precision + Recall).

CHAPTER 9
Deployment

9.1 Steps to deploy

1. Create a GitHub repository and push the project code, `requirements.txt` and sample data.
2. Create a virtual environment and install the dependencies with `pip install -r requirements.txt`.
3. Run the application locally with `streamlit run app.py` and verify all pages.
4. Create the database (SQLite file or MySQL schema) and set the connection string.
5. Log in to Streamlit Community Cloud, select the repository, branch and `app.py`, and deploy.
6. Add the secrets (database URL, login credentials) in the hosting settings.
7. Optionally publish the Power BI report to Power BI Service and share the link.

9.2 Environment configuration

```
# requirements.txt
pandas
numpy
scikit-learn
xgboost
matplotlib
seaborn
streamlit
sqlalchemy
pymysql
joblib
openpyxl


# .streamlit/secrets.toml  (not committed to GitHub)
DB_URL = "sqlite:///retail.db"
# DB_URL = "mysql+pymysql://user:password@host/retail"
```

9.3 Hosting of front-end and back-end

Because the front-end and the back-end are part of the same Streamlit application, they are hosted together: on a local machine or office server for

private use, or on Streamlit Community Cloud for browser access from anywhere. The database is a file (SQLite) bundled with the host or an external MySQL server reachable from the host. The optional Power BI report is hosted separately on Power BI Service.

CHAPTER 10
Challenges & Limitations

As the project is at the design stage (Review-1), this chapter lists the anticipated challenges and the solutions adopted in the design; it will be updated with actual issues as implementation proceeds.

10.1 Issues faced during development (anticipated) and 10.2 Solutions applied

Issue	Solution applied in the design
Raw data is unorganised, with missing values, duplicates and outliers	Systematic pre-processing module and data-profiling report before any analysis
Features on different scales distort K-Means	Standardisation with `StandardScaler` before clustering
Choosing the number of clusters K	Elbow method supported by silhouette score; labels checked against business meaning
Imbalanced classes in purchase prediction	Stratified split, class weights and F1-score/Recall instead of Accuracy alone
Information leakage between features and target	Features built from an earlier window, target from the following period
Non-technical users cannot interpret model output	Plain-language segment names, action lists and a simple dashboard
Different datasets use different column names	Column validation and a mapping step during upload

Table 10.1 Challenges and solutions

10.3 Current limitations or known bugs

- Quality of the segments and predictions depends on the quality and size of the supplied dataset.
- The system works on historical (batch) data; it is not connected to live POS data.
- New customers with little history (cold start) cannot be predicted reliably.
- Seasonality and external factors (promotions, holidays, competition) are not modelled in the first version.
- K-Means assumes roughly spherical clusters and needs K to be chosen; Fuzzy C-Means (reported better silhouette in [3]) is computationally heavier.

CHAPTER 11
Future Enhancements

11.1 Planned features

- Add further behavioural dimensions such as Length and Volume (LRFMV) [1] and Time (RFMT) [2] to improve segmentation.
- Customer lifetime value (CLV) and churn-probability modelling.
- Product recommendation and market-basket analysis for each segment.
- Seasonality-aware forecasting of sales and customer purchases.
- Comparison with hierarchical and Fuzzy C-Means clustering [3].
- Multi-store and multi-user support with richer role management.

11.2 Possible integrations or optimizations

- Integration with POS / e-commerce systems for automatic, near real-time data loading.
- Automatic e-mail/SMS/WhatsApp campaign tools connected to the action lists.
- REST API (FastAPI) so that other applications can request predictions.
- Containerisation with Docker and scheduled retraining of the models.
- Hyper-parameter tuning, model explainability (feature importance, SHAP) and performance optimisation for very large datasets.

CHAPTER 12
Conclusion

12.1 Summary of the project

Smart Retail Customer Analysis & Prediction converts raw, unorganised retail transaction data into structured, analysis-ready information, segments customers with K-Means clustering and predicts future purchase likelihood and customer value using Logistic Regression, Random Forest and XGBoost. The results are made available to retailers through an interactive Streamlit / Power BI dashboard. By combining segmentation and prediction in one pipeline and by presenting action-oriented outputs, the system aims to improve customer engagement, personalised marketing and sales planning, resulting in a smarter and more efficient retail-management system.

12.2 What was achieved

Up to Project Review-1 (17 September 2026) the following has been achieved: the problem was defined and the objectives were set; a literature survey of three recent RFM-based studies was completed and the research gap (narrow features, segmentation without prediction, tools unsuited to small retailers, no unified interpretable output) was identified; the novelty of the approach was formulated; the end-to-end design flow, technology stack, database design and dashboard wireframes were prepared; and a 13-week project plan was drawn up. Implementation, model training, evaluation and testing follow in Phases 2 to 6, and the results of these phases will be added to this report.

12.3 Skills learned during development

- Data cleaning, profiling and feature engineering with Pandas and NumPy.
- Unsupervised learning (K-Means, Elbow method) and supervised learning (Logistic Regression, Random Forest, XGBoost) with Scikit-learn.
- Model evaluation with Accuracy, Precision, Recall and F1-score.
- Data visualisation with Matplotlib and Seaborn; dashboard building with Streamlit / Power BI.
- Relational database design and access with SQL (MySQL/SQLite).

- Literature review, report writing, teamwork and version control with Git/GitHub.

CHAPTER 13
References

1. Mahfuza et al., "LRFMV: An Efficient Customer Segmentation Model for Superstores," PLOS ONE, 2024.
2. Ullah et al., "Customer Analysis Using ML-Based Classification for Segmentation Using RFMT," Sensors, 2025.
3. Chavhan et al., "RFM Analysis for Customer Segmentation Using Machine Learning: A Survey of a Decade of Research," 2026.
4. Scikit-learn documentation (clustering, ensemble methods, model evaluation): https://scikit-learn.org
5. Pandas documentation: https://pandas.pydata.org/docs
6. NumPy documentation: https://numpy.org/doc
7. XGBoost documentation: https://xgboost.readthedocs.io
8. Matplotlib documentation: https://matplotlib.org and Seaborn documentation: https://seaborn.pydata.org
9. Streamlit documentation: https://docs.streamlit.io
10. SQLite documentation: https://www.sqlite.org/docs.html and MySQL Reference Manual: https://dev.mysql.com/doc

CHAPTER 14

Appendices

A. Screenshots of the app

Screenshots of the Upload, Data Profile, Segments, Predictions and Insights pages will be inserted here after the dashboard is developed (Phase 5). The planned layout of these pages is given in the wireframes of Figure 5.1.

B. Sample code snippets

B.1 RFM feature engineering

```
import pandas as pd

df = pd.read_csv("transactions.csv", parse_dates=["InvoiceDate"])
df = df.dropna(subset=["CustomerID"]).drop_duplicates()
df = df[(df["Quantity"] > 0) & (df["UnitPrice"] > 0)]
df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

snapshot = df["InvoiceDate"].max() + pd.Timedelta(days=1)
rfm = df.groupby("CustomerID").agg(
    Recency=("InvoiceDate", lambda d: (snapshot - d.max()).days),
    Frequency=("InvoiceNo", "nunique"),
    Monetary=("TotalAmount", "sum"),
    CategoryDiversity=("ProductCategory", "nunique"),
)
```

B.2 K-Means segmentation with the Elbow method

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

cols = ["Recency", "Frequency", "Monetary"]
X = StandardScaler().fit_transform(rfm[cols])

for k in range(2, 9):                       # Elbow / silhouette check
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_)
    print(k, round(km.inertia_, 1), round(sil, 3))

best_k = 4                                  # chosen from the Elbow curve
rfm["Cluster"] = KMeans(n_clusters=best_k, n_init=10,
                        random_state=42).fit_predict(X)
```

B.3 Training and comparing the prediction models

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

# features: RFM table; target: 1 if customer buys in the next period
X_tr, X_te, y_tr, y_te = train_test_split(
    features, target, test_size=0.2, stratify=target, random_state=42)
```

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000,
                                              class_weight="balanced"),
    "Random Forest": RandomForestClassifier(n_estimators=200,
                                            class_weight="balanced"),
    "XGBoost": XGBClassifier(eval_metric="logloss"),
}
for name, model in models.items():
    pred = model.fit(X_tr, y_tr).predict(X_te)
    print(name, accuracy_score(y_te, pred), precision_score(y_te, pred),
          recall_score(y_te, pred), f1_score(y_te, pred))
```

C. Installation / setup instructions

1. Install Python 3.10 or later and Git.
2. Clone the repository and open the project folder.
3. Create and activate a virtual environment (`python -m venv venv`).
4. Install the dependencies: `pip install -r requirements.txt`.
5. (Optional) Install MySQL and create a database named `retail`; otherwise SQLite is used automatically.
6. Start the application: `streamlit run app.py` and open the address shown in the terminal.

D. User manual or guide

1. Open the dashboard in the browser and log in.
2. On **Upload Data**, choose the transaction file (CSV/Excel) and check the preview.
3. On **Data Profile**, review missing values and data types, then press **Clean data**.
4. On **RFM Features**, press **Generate features** to see the customer table.
5. On **Segments**, choose the number of clusters (guided by the Elbow plot) and view the segment names and sizes.
6. On **Predictions**, select a model and view purchase probabilities and the model comparison.
7. On **Insights**, select a segment and download the promotion list, at-risk alerts or high-value flags.

E. Certifications

Course / workshop certificates of the team members to be attached here.